Robot Learning Engineer Master Curriculum

Day 3 Packet - July 7, 2026

Target Role: Robot Learning / Embodied AI Engineer

Today converts simple programs into reusable logic. You will learn loops, functions, lists, and the first AI-data habit: representing sensor data as arrays. This is the bridge from beginner Python to robotics and machine learning code.

Day Overview

Block	Time	Focus	Main Output
1	2 hours	Python loops + functions	6 reusable robotics-flavored programs
2	1.25 hours	DSA arrays/strings	Valid Anagram and Best Time to Buy/Sell Stock
3	1 hour	NumPy first touch	Sensor-array notebook/script completed
4	1 hour	Robotics map	ROS 2 concepts note: nodes, topics, messages
5	35 minutes	GitHub cleanup	Folder structure, README update, commit
6	10 minutes	Journal	Day03 report written

Primary Resources for Today

Use only the resources below today. Do not jump between random tutorials. Resource discipline is part of the training.

Track	Resource	Use Today
Python course	CS50P Week 2: Loops	Watch/read Loops. Focus on for, while, range, break, and input validation loops.
Python documentation	Python control flow tools	Read for, range, break/continue, and defining functions.
AI data foundation	NumPy Quickstart	Read only prerequisites, basics, ndarray shape, ndim, size, dtype, and array creation.
DSA practice	LeetCode Valid Anagram	Practice character counts/hash maps.
DSA practice	LeetCode Best Time to Buy and Sell Stock	Practice single-pass array scan and maintaining best-so-far value.
Robotics docs	ROS 2 Humble documentation	Do a conceptual skim only: what are nodes, topics, messages, and packages?

Book note: Use legal access only. If a paid book is unavailable, use the official course/documentation and complete the code deliverables.

Block 1 - Python Loops, Functions, and Lists

Goal: Stop writing one-time scripts. Start writing reusable logic.

- Study CS50P Week 2: Loops.
- From Python docs, read for statements, range(), break/continue, and defining functions.
- Every program today must use at least one function.
- At least three programs must use a list.

File	Program Requirement
sensor_average.py	Input N sensor readings into a list and print average, min, max.
battery_drain_simulator.py	Simulate battery percentage decreasing every time step until critical.
weld_speed_monitor.py	Input multiple travel-speed readings and count good/slow/fast readings.
ctwd_series_analyzer.py	Analyze a list of CTWD values and report how many are ideal.
path_points_generator.py	Generate x-y path points for a simple straight-line robot motion.
function_calculator.py	Rewrite calculator using functions: add, subtract, multiply, divide.

```
robot-learning-journey/
  Day03/
    sensor_average.py
    battery_drain_simulator.py
    weld_speed_monitor.py
    ctwd_series_analyzer.py
    path_points_generator.py
    function_calculator.py
    dsa_notes.md
    numpy_sensor_arrays.py
    ros2_concepts.md
    journal.md
```

Block 2 - DSA Arrays and Strings

Goal: Build interview logic slowly but correctly.

Do not chase many problems. Two deeply explained problems are better than ten copied solutions.

Problem	Required Approach
Valid Anagram	Write sorting idea and hash-map/count idea. Explain O(n log n) vs O(n).
Best Time to Buy and Sell Stock	Use one pass. Track minimum price so far and best profit so far.
Optional: Move Zeroes	Use this only if the first two are comfortable. Practice in-place thinking.

Block 3 - NumPy First Touch: Sensor Arrays

Goal: Understand why AI/robotics data is represented as arrays/tensors.

Read the NumPy Quickstart sections on ndarray basics, shape, ndim, size, dtype, and array creation. Then create numpy_sensor_arrays.py.

- Create a NumPy array of 10 mock distance sensor readings.
- Print shape, ndim, size, dtype, mean, min, and max.
- Create a 2D array representing 5 robot poses: x, y, theta.
- Print the first pose, last pose, and mean x/y position.
- Write 5 lines explaining why NumPy helps AI/robotics.

```
# numpy_sensor_arrays.py
import numpy as np

readings = np.array([0.32, 0.35, 0.31, 0.40, 0.38])
print(readings.shape, readings.mean(), readings.min(), readings.max())
```

Block 4 - Robotics Concept Map: ROS 2 Basics

Goal: Understand ROS 2 language before installing or coding deep ROS projects.

Create ros2_concepts.md. Write simple explanations. Do not install anything today unless your environment is already ready.

- What is ROS 2?
- What is a node?
- What is a topic?
- What is a message?
- What is a package?
- Why do robot systems need middleware?

Block 5 - GitHub Cleanup and Commit

Goal: Make your repository readable to another person.

Update README.md with a Week 1 progress table. Add Day01, Day02, Day03 links or checklist status if possible.

```
git status
git add .
git commit -m "Day 3: Python loops functions arrays and NumPy basics"
git push
```

Completion Checklist

```
[ ] CS50P Week 2 Loops watched/read.
[ ] Python docs control-flow sections reviewed: for, range, break/continue, functions.
[ ] 6 Python programs completed in Day03 folder.
[ ] Valid Anagram and Best Time to Buy/Sell Stock attempted with notes.
[ ] numpy_sensor_arrays.py completed and run successfully.
[ ] ros2_concepts.md completed in simple language.
[ ] README.md updated with Week 1 progress table.
[ ] Day03 Git commit created and pushed.
[ ] journal.md completed and progress pasted into chat.
```

What To Send Me After Finishing

Copy-paste this report into chat. This is how we adjust the next day without losing context.

```
Day 3 completed report:
1. Hours studied:
2. Python files completed:
3. DSA problems completed:
4. NumPy task completed: yes/no
5. ROS 2 notes completed: yes/no
6. GitHub link/commit:
7. What was easy:
8. What was hard:
9. What I skipped:
10. Questions:
```

Quality Rules

- Do not copy code without understanding it yourself.
- Do not hide skipped tasks. Honest reporting makes the roadmap accurate.
- Every study day ends with code, notes, a GitHub commit, and a journal entry.
- Certificates are secondary. GitHub evidence, projects, research-quality work, and interview skill matter more.

Source Notes and Resource Policy

Day 3 introduces NumPy because robot learning work eventually becomes tensor operations: camera pixels, depth maps, joint states, poses, trajectories, and model inputs. We start with simple sensor arrays before deep learning.